

INTELLIGENT LEARNING PLATFORM

Technologie- Dokumentation

Welche Komponenten die ILP einsetzt, wofür sie da sind, wie sie gesichert werden und was sie sicher macht.

Version	1.0
Stand	19.08.2026
Verantwortlich	Reza Nazarian
Klassifizierung	Intern

Überblick

Die ILP besteht aus wenigen, weit verbreiteten Standard-Technologien. Keine der eingesetzten Komponenten ist exotisch oder proprietär — alle sind Open Source mit großer Verbreitung, aktiver Pflege und etablierten Betriebsverfahren.

Dieses Dokument beschreibt für jede Komponente vier Punkte: **was sie tut, wie sie in der ILP eingebunden ist, wie Backup und Restore funktionieren und welche Sicherheitsmaßnahmen greifen.**

DIE KOMPONENTEN AUF EINEN BLICK

Komponente	Art	Aufgabe in der ILP	Hält Daten?
Ruby on Rails	Web-Framework	Backend von Content Factory und Content Delivery: Fachlogik, Benutzer, Rechte, API	Nein (Daten liegen in der PostgreSQL der Group-IT)
FastAPI AI-Backend	Service	Alles rund um KI: Dokumente auswerten, Kursinhalte generieren, Chatbot	Nein
React	Frontend-Framework	Oberflächen für Learning Designer, Lernende und Lernbegleiter	Nein (läuft im Browser)
Celery	Worker Queue	Führt die langlaufenden KI-Aufträge im FastAPI AI-Backend im Hintergrund aus	Nur laufende Aufträge
Sidekiq	Worker	Hintergrundjobs der Content Factory (Import, Generierungs-Steuerung, ERP-Sync)	Nur laufende Jobs
SolidQueue	Worker	Hintergrundjobs der Content Delivery (Fortschritt, Gamification, E-Mails)	Jobs in der PostgreSQL der Group-IT
Dragonfly DB	In-Memory-Speicher	Zwischenspeicher und Job-Vermittlung zwischen Backend und Workern	Nur kurzlebig
GitHub	Plattform	Quellcode-Verwaltung, Code-Reviews, automatisierte Tests und Auslieferung	Quellcode, kein Kundendatenbestand

WIE DIE KOMPONENTEN ZUSAMMENSPIELEN

Ein Digital Learning Designer arbeitet im **React**-Frontend der Content Factory. Seine Eingaben gehen an das **Rails**-Backend, das die Fachlogik ausführt und in Postgres speichert. Dauert etwas länger — etwa eine Kursgenerierung — legt Rails einen Auftrag über **Sidekiq** ab und antwortet der Oberfläche sofort. Der Auftrag geht an das **FastAPI AI-Backend**, das ihn über **Celery** abarbeitet und mit OpenAI kommuniziert. **Dragonfly** ist dabei der schnelle Briefkasten, über den die Aufträge vermittelt werden. Auf der Lernenden-Seite liefert die Content Delivery Kurse über ein zweites React-Frontend aus; ihre Hintergrundaufgaben laufen über **SolidQueue**. Der gesamte Quellcode und die Auslieferung laufen über **GitHub**.

BETRIEB UND VERANTWORTLICHKEITEN

Die PostgreSQL-Datenbank, in der alle fachlichen und personenbezogenen Daten der ILP liegen, wird **nicht von edutech, sondern von der Group-IT betrieben** und als Datenbank-Service bereitgestellt. Für Verfügbarkeit, Härtung, Patching, Verschlüsselung sowie Backup und Restore der Datenbank gelten damit die etablierten Standards und Verfahren des Konzerns. edutech verantwortet ausschließlich Datenmodell und Anwendungslogik.

Bereich	Betrieb durch	Verantwortung von edutech
PostgreSQL-Datenbank	Group-IT	Datenmodell, Migrationen, Rechtevergabe innerhalb der Anwendung, Beauftragung von Restores
Anwendungen und Worker (Rails, FastAPI, React, Celery, Sidekiq, SolidQueue)	edutech	Entwicklung, Auslieferung, Betrieb, Monitoring
Dragonfly DB und Object Storage	edutech	Betrieb, Konfiguration, Snapshots
GitHub	Konzern-Enterprise-Account	Repositories, Reviews, Build- und Deployment-Pipelines

Grundprinzip für Backups: Anwendungen und Worker sind austauschbar — sie enthalten keine Daten und werden bei einem Ausfall einfach neu aus dem letzten freigegebenen Stand ausgerollt. Gesichert werden muss nur, was Zustand hat: die Datenbank, der Object Storage und der Quellcode. Der mit Abstand wichtigste Teil davon — die Datenbank — wird durch die Group-IT nach Konzernstandard gesichert.

1 Backend mit Ruby on Rails

Content Factory und Content Delivery · Open Source · seit 2004 etabliert

WAS DIE KOMPONENTE MACHT

Ruby on Rails ist ein seit über 20 Jahren verbreitetes Web-Framework (u. a. GitHub, Shopify, Airbnb). Es bildet in der ILP das Herzstück beider Anwendungen: Es nimmt die Anfragen der Oberfläche entgegen, prüft, ob der Benutzer angemeldet und berechtigt ist, führt die Fachlogik aus und liest bzw. schreibt die Daten in die von der Group-IT betriebene PostgreSQL-Datenbank.

- **Content Factory:** Kursstrukturen, Storyboards, Freigabe-Workflows, Benutzer- und Rollenverwaltung.
- **Content Delivery:** Kursauslieferung an Lernende, Fortschritt, Quiz-Auswertung, Gamification.

WIE IN DIE ILP INTEGRIERT

- Stellt eine JSON-API bereit, die ausschließlich von den React-Frontends genutzt wird.
- Speichert alle fachlichen Daten in der PostgreSQL der Group-IT — Rails selbst hält keine Daten dauerhaft. Die Anbindung erfolgt über ein eigenes technisches Konto mit eng gefassten Rechten.
- Übergibt langlaufende Arbeit an die Worker (Sidekiq in der Factory, SolidQueue in der Delivery) und ruft für KI-Aufgaben das FastAPI AI-Backend auf.
- Nutzt Dragonfly als Zwischenspeicher für Sitzungen, Caching und Job-Vermittlung.
- Übernimmt Stammdaten aus dem ERP über das Learning Management (rein lesend).

BACKUP & RESTORE

- **Zu sichern ist nichts an der Anwendung selbst** — sie ist zustandslos. Der ausgelieferte Stand liegt als Container-Image in der GitHub-Registry, der Quellcode in GitHub.
- **Die Datensicherung der PostgreSQL verantwortet die Group-IT** im Rahmen ihres Datenbank-Betriebs: Vollsicherung plus fortlaufendes Transaktionslog (Point-in-Time-Recovery), verschlüsselte Ablage und Aufbewahrung nach Konzernvorgabe. edutech betreibt hierfür kein eigenes Verfahren, sondern lässt sich Umfang, Frequenz und Aufbewahrung bestätigen.
- Hochgeladene Dateien (Quelldokumente, Bilder) liegen im Object Storage mit eigener Versionierung.
- **Restore:** Anwendungsseitig rollt edutech das Container-Image der letzten freigegebenen Version erneut aus und spielt Konfiguration und Schlüssel aus dem Secret-Store ein — das dauert Minuten. Die Rücksicherung der Datenbank auf einen gewünschten Zeitpunkt wird über den definierten Meldeweg bei der Group-IT beauftragt und von dieser durchgeführt; die Gesamtdauer richtet sich damit nach deren Wiederherstellungszeiten.
- Der Restore-Ablauf wird gemeinsam mit der Group-IT halbjährlich in einer separaten Umgebung geprobt und protokolliert — inklusive der Schnittstelle zwischen Datenbank-Rücksicherung und Wiederanlauf der Anwendung.

SICHERHEITSASPEKTE

- Rails bringt Standardschutz gegen die gängigen Web-Angriffe von Haus aus mit: CSRF-Token, Schutz vor SQL-Injection durch parametrisierte Abfragen, Schutz vor ungewollter Massenzuweisung von Feldern.
- Anmeldung mit bcrypt-gehashten Passwörtern, HttpOnly-Session-Cookies, Sitzungsende nach 8 Stunden Inaktivität, Sperre nach zu vielen Fehlversuchen.
- Rollenbasierte Rechteprüfung an jeder Schnittstelle — insbesondere die Trennung, dass Lernbegleiter keine ILA-Konversationen sehen können.

- Zugriffe auf personenbezogene Daten werden in einem revisionssicheren Audit-Log protokolliert.
- Härtung, Patching und Zugriffskontrolle auf Datenbankebene liegen bei der Group-IT und folgen dem Konzernstandard; die Anwendung erreicht die Datenbank ausschließlich über das interne Netz.
- Zugangsdaten liegen nie im Code, sondern verschlüsselt in den Rails-Credentials bzw. im Secret-Store der Umgebung.
- Sicherheitslücken in Abhängigkeiten werden automatisiert über `bundler-audit` und Dependabot gemeldet, statische Codeprüfung mit Brakeman läuft bei jedem Pull Request.
- Verschlüsselung: TLS 1.3 im Transport, AES-256 auf den Datenträgern.

2 AI-Backend mit FastAPI

Kursgenerierung, Bewertung und Intelligent Learning Assistant

WAS DIE KOMPONENTE MACHT

Das AI-Backend ist ein eigenständiger Dienst auf Basis von FastAPI, dem heute verbreitetsten Python-Framework für APIs, und bündelt alle KI-nahen Aufgaben. Es liest die hochgeladenen Quelldokumente, erzeugt daraus Grobkonzept, Feinkonzept und Storyboards, bewertet Freitext- und Code-Antworten und betreibt den sokratischen Dialog des Intelligent Learning Assistant. Die eigentliche Sprachmodell-Leistung kommt von OpenAI; das AI-Backend ist die kontrollierte Schleuse davor.

WIE IN DIE ILP INTEGRIERT

- Wird ausschließlich intern von den Rails-Backends aufgerufen — es ist nicht aus dem Internet erreichbar.
- Nimmt Aufträge entgegen, stellt sie in die Celery-Queue und meldet das Ergebnis an Rails zurück.
- Alle Prompt-Vorlagen liegen versioniert in GitHub, nicht in der Datenbank — Änderungen sind damit nachvollziehbar und überprüfbar.
- Ein- und ausgehende Prompts werden für Nachvollziehbarkeit und Qualitätssicherung gespeichert (verschlüsselt, 90 Tage).

BACKUP & RESTORE

- Der Dienst ist zustandslos: Code in GitHub, ausgeliefertes Container-Image in der Registry, keine eigene Datenhaltung.
- Ergebnisse und Protokolle liegen in der PostgreSQL und sind damit vom Datenbank-Backup der Group-IT abgedeckt.
- **Restore:** Image neu ausrollen, API-Schlüssel aus dem Secret-Store beziehen, Dienst ist sofort wieder arbeitsfähig. Unterbrochene Generierungsläufe werden anhand ihres in der Datenbank gespeicherten Status neu angestoßen — es geht kein Auftrag verloren, es entsteht höchstens eine erneute Generierung.
- Prompt-Vorlagen lassen sich über GitHub jederzeit auf einen früheren Stand zurücksetzen.

SICHERHEITSASPEKTE

- **Datensparsamkeit gegenüber OpenAI:** An das Sprachmodell gehen Kursinhalte und der fachliche Kontext, aber keine Klarnamen, keine Personalnummern und keine E-Mail-Adressen. Der Drittlandtransfer ist im Datenverzeichnis (DK-023, DK-070 ff.) dokumentiert und über einen Auftragsverarbeitungsvertrag abgedeckt.
- **Prompt Injection:** Inhalte aus Quelldokumenten und Lernenden-Eingaben werden als Daten und nicht als Anweisung behandelt; Systemanweisungen sind davon getrennt. Antworten werden vor der Weiterverarbeitung strukturell geprüft.
- **Hochgeladene PDFs** werden als potenziell unsicher behandelt: Größen- und Typprüfung, Verarbeitung in einem abgeschotteten Prozess ohne Netzwerkzugriff nach außen.
- **Code aus Code-Quizen** wird nicht ausgeführt, sondern ausschließlich durch das Sprachmodell bewertet — damit entfällt das Risiko der Codeausführung auf unseren Systemen.
- Datenaustausch ausschließlich als JSON; unsichere Python-Serialisierung (`pickle`) ist abgeschaltet.
- Abhängigkeiten sind über Lockfiles fixiert und werden automatisiert auf bekannte Schwachstellen geprüft.

3 Frontend mit React

Oberflächen von Content Factory und Content Delivery

WAS DIE KOMPONENTE MACHT

React ist die von Meta entwickelte, heute meistgenutzte Bibliothek für Web-Oberflächen. In der ILP gibt es zwei Frontends: die Arbeitsumgebung der Digital Learning Designer und Subject Matter Experts in der Content Factory und die Lernumgebung für Lernende und Lernbegleiter in der Content Delivery. Beide laufen als reine Web-Anwendung im Browser — es ist keine Installation auf den Endgeräten nötig.

WIE IN DIE ILP INTEGRIERT

- Wird als statisches Paket aus HTML, JavaScript und CSS ausgeliefert und kommuniziert ausschließlich über die JSON-API mit dem jeweiligen Rails-Backend.
- Enthält keine Fachlogik mit Sicherheitsrelevanz: Jede Berechtigung wird serverseitig erneut geprüft; die Oberfläche blendet lediglich aus, was der Benutzer ohnehin nicht darf.
- Erfasst die Lernaktivitäts-Ereignisse (Verweildauer, Scrolltiefe, Sichtbarkeit des Tabs) und meldet sie gebündelt an das Backend.

BACKUP & RESTORE

- Im Frontend liegen keine dauerhaften Daten — es gibt nichts zu sichern außer dem Quellcode, der in GitHub liegt.
- Jeder ausgelieferte Stand ist ein versioniertes Build-Artefakt; die exakten Abhängigkeitsversionen sind über Lockfiles festgehalten, jeder Stand ist also reproduzierbar neu baubar.
- **Restore:** Das Artefakt der letzten funktionierenden Version erneut ausrollen — in wenigen Minuten erledigt. Ein Rückfall auf eine ältere Version ist jederzeit möglich, ohne Datenrisiko.

SICHERHEITSASPEKTE

- **Kein Zugriff auf Daten ohne Backend:** Das Frontend hat keine eigenen Datenbankzugänge und keine API-Schlüssel; alles läuft über die authentifizierte Sitzung.
- Session-Token liegen in HttpOnly-Cookies und nicht im Browser-Speicher — damit sind sie für Skripte im Browser nicht auslesbar.
- **Schutz vor Cross-Site-Scripting:** React maskiert Ausgaben standardmäßig. KI-generierte Kursinhalte werden zusätzlich serverseitig auf erlaubte Auszeichnungen reduziert, bevor sie im Browser dargestellt werden.
- Eine Content-Security-Policy schränkt ein, aus welchen Quellen Skripte und Inhalte geladen werden dürfen.
- npm-Abhängigkeiten werden fixiert, automatisiert auf Schwachstellen geprüft und bewusst schlank gehalten.
- Lokal im Browser werden nur unkritische Anzeigeeinstellungen abgelegt, keine personenbezogenen Inhalte.

4 Celery Worker Queue

Hintergrundverarbeitung im FastAPI AI-Backend

WAS DIE KOMPONENTE MACHT

Celery ist der Standard-Mechanismus in der Python-Welt, um Arbeit im Hintergrund abzuarbeiten. Eine Kursgenerierung dauert Minuten bis Stunden — niemand soll währenddessen im Browser warten. Celery nimmt solche Aufträge entgegen, arbeitet sie der Reihe nach ab und meldet das Ergebnis zurück. Eine eigene Fachlogik hat Celery nicht; es ist reine Ablaufsteuerung.

WIE IN DIE ILP INTEGRIERT

- Läuft ausschließlich innerhalb des FastAPI AI-Backends und verarbeitet dessen Aufgaben: Dokumente aufbereiten, Grob- und Feinkonzepte erzeugen, Storyboards generieren, Bilder erzeugen, Antworten bewerten.
- Als Vermittler (Broker) dient Dragonfly; der Fortschritt eines Auftrags wird zusätzlich in PostgreSQL geführt, damit er auch nach einem Neustart nachvollziehbar bleibt.
- Getrennte Warteschlangen für schnelle und langlaufende Aufträge, damit ein großer Generierungslauf den Chatbot nicht ausbremst.

BACKUP & RESTORE

- Die Warteschlange selbst wird nicht gesichert — sie enthält nur Arbeitsaufträge weniger Minuten oder Stunden, keine Bestandsdaten.
- Die Absicherung erfolgt fachlich: Jeder Auftrag hat einen Status in der Datenbank. Nach einem Ausfall wird anhand dieses Status erkannt, was offen ist, und erneut eingestellt.
- Aufträge sind so gebaut, dass eine erneute Ausführung unschädlich ist; die Bestätigung erfolgt erst nach erfolgreichem Abschluss, ein abgebrochener Auftrag geht damit nicht verloren.
- **Restore:** Worker-Container neu starten. Dauerhaft fehlschlagende Aufträge landen in einer Fehler-Warteschlange und werden manuell gesichtet.

SICHERHEITSASPEKTE

- Kein Zugang von außen: Worker sind nur im internen Netz erreichbar und nehmen Aufträge ausschließlich aus dem eigenen Broker entgegen.
- Auftragsdaten werden ausschließlich als JSON übertragen; die in Celery grundsätzlich mögliche, unsichere `pickle`-Serialisierung ist deaktiviert. Damit ist das bekannteste Risiko dieser Technologie ausgeschlossen.
- Die Verbindung zum Broker ist mit Passwort und TLS abgesichert.
- In den Aufträgen stehen Referenzen (IDs), keine personenbezogenen Klartextdaten.
- Zeit- und Speicherlimits pro Auftrag verhindern, dass ein einzelner Lauf die Umgebung blockiert.

5 Sidekiq Worker

Hintergrundjobs der Content Factory (Rails)

WAS DIE KOMPONENTE MACHT

Sidekiq ist das seit Jahren etablierte Werkzeug, um in Rails-Anwendungen Aufgaben im Hintergrund auszuführen. In der Content Factory übernimmt es alles, was länger dauert als eine normale Webanfrage: Quelldokumente entgegennehmen und aufbereiten, Generierungsläufe beim AI-Backend anstoßen und überwachen, Stammdaten aus dem ERP synchronisieren, Benachrichtigungen versenden.

WIE IN DIE ILP INTEGRIERT

- Läuft als eigener Prozess neben dem Rails-Backend der Content Factory und teilt sich mit ihm Code und Datenbankzugang.
- Nutzt Dragonfly als Warteschlange; Rails stellt einen Job ein und antwortet der Oberfläche sofort.
- Ein Verwaltungs-Dashboard zeigt laufende, wartende und fehlgeschlagene Jobs.
- Fehlgeschlagene Jobs werden mit wachsenden Abständen automatisch wiederholt.

BACKUP & RESTORE

- Der Worker selbst ist zustandslos und wird wie das Rails-Backend aus dem Container-Image neu ausgerollt.
- Die Warteschlange wird nicht gesichert; sie ist kurzlebig. Maßgeblich ist der fachliche Zustand in der PostgreSQL, der über das Datenbank-Backup der Group-IT abgedeckt ist.
- Jobs sind idempotent und werden erst nach erfolgreicher Ausführung als erledigt markiert; ein Absturz führt zur Wiederholung, nicht zum Verlust.
- **Restore:** Worker-Prozesse neu starten. Für dauerhaft fehlgeschlagene Jobs existiert eine Fehlerliste, aus der gezielt nachgestartet werden kann. Beim Herunterfahren beenden die Worker laufende Jobs geordnet.

SICHERHEITSASPEKTE

- **Das Sidekiq-Dashboard ist die einzige Web-Oberfläche der Komponente** und ist nicht öffentlich erreichbar: Zugriff nur nach Anmeldung mit Administratorrolle und nur aus dem internen Netz.
- In Job-Parametern stehen nur IDs, keine personenbezogenen Klartextdaten — damit tauchen keine Namen oder Antworten in Warteschlangen oder Fehlerlisten auf.
- Der Zugriff auf Dragonfly ist mit Passwort und TLS abgesichert und auf das interne Netz beschränkt.
- Der Worker hat dieselben, eng gefassten Datenbankrechte wie die Anwendung — keine erweiterten Berechtigungen.
- Fehler werden an das Monitoring (Sentry, EU-Region) gemeldet, ohne personenbezogene Nutzdaten zu übertragen.

6 SolidQueue Worker

Hintergrundjobs der Content Delivery (Rails)

WAS DIE KOMPONENTE MACHT

SolidQueue ist der seit Rails 8 mitgelieferte Standard für Hintergrundjobs. Der Unterschied zu Sidekiq: Die Warteschlange liegt nicht in einem separaten Speicher, sondern direkt in der von der Group-IT betriebenen PostgreSQL. In der Content Delivery erledigt SolidQueue die Aufgaben rund um den laufenden Lernbetrieb — Fortschritt verrechnen, Gamification-Punkte und Badges vergeben, Lernereignisse an die Lerndatenmessung übergeben, Erinnerungs-E-Mails versenden.

WIE IN DIE ILP INTEGRIERT

- Läuft als eigener Prozess neben dem Rails-Backend der Content Delivery.
- Speichert Jobs in eigenen Tabellen derselben PostgreSQL-Datenbank — es wird keine zusätzliche Infrastruktur und kein weiterer Dienst benötigt.
- Job und Fachdaten werden in derselben Transaktion geschrieben: Ein Job existiert nur, wenn die zugehörige fachliche Änderung tatsächlich gespeichert wurde.
- Bewusste Trennung von der Content Factory: Der Lernbetrieb ist von Generierungslasten unabhängig.

BACKUP & RESTORE

- **Besonderheit:** Da die Jobs in der PostgreSQL liegen, sind sie automatisch Teil des von der Group-IT verantworteten Datenbank-Backups — ein separates Sicherungsverfahren entfällt.
- Nach einem Neustart nimmt der Worker die offenen Jobs aus der Tabelle selbstständig wieder auf; abgebrochene Jobs werden über einen Zeitablauf erkannt und erneut eingeplant.
- **Restore:** Worker-Prozess neu starten. Setzt die Group-IT die Datenbank auf einen früheren Zeitpunkt zurück, werden die Warteschlangen-Tabellen vor dem Start der Worker bereinigt, damit keine veralteten Jobs erneut ausgeführt werden. Dieser Schritt ist fester Bestandteil der Restore-Anleitung und muss mit der Freigabe der Datenbank durch die Group-IT abgestimmt sein.
- Erledigte Jobs werden nach kurzer Aufbewahrung automatisch gelöscht.

SICHERHEITSASPEKTE

- **Kleinste Angriffsfläche aller Worker:** keine zusätzliche Netzwerkverbindung, kein zusätzlicher Dienst, keine eigene Weboberfläche — nur die ohnehin vorhandene Datenbankverbindung.
- Job-Daten liegen in der Datenbank und unterliegen damit denselben Schutzmaßnahmen wie alle übrigen Daten — einschließlich der Verschlüsselung, Zugriffskontrolle und Sicherung durch die Group-IT.
- Auch hier enthalten Job-Parameter nur IDs statt personenbezogener Klartextdaten.
- Wiederholungsgrenzen verhindern, dass ein dauerhaft fehlschlagender Job das System belastet.

7 Dragonfly DB

Zwischenspeicher und Job-Vermittlung

WAS DIE KOMPONENTE MACHT

Dragonfly ist ein moderner In-Memory-Datenspeicher — ein direkt kompatibler Ersatz für Redis mit deutlich besserer Speicher- und Kernauslastung. Er hält Daten im Arbeitsspeicher und ist damit sehr schnell. In der ILP ist Dragonfly kein Datenbestand, sondern ein Beschleuniger und Briefkasten: Er speichert kurzlebige Dinge und vermittelt Aufträge zwischen Anwendungen und Workern.

WIE IN DIE ILP INTEGRIERT

- Warteschlange für Sidekiq (Content Factory) und Broker für Celery (AI-Backend).
- Sitzungsspeicher und Cache für häufig gelesene, unkritische Daten wie Kursstrukturen.
- Zähler für Anmeldeversuche und Rate Limiting.
- Alle Einträge haben eine Ablaufzeit — es entsteht kein dauerhafter Datenbestand.

BACKUP & RESTORE

- Dragonfly wird bewusst als **verlierbar** betrachtet: Es enthält keine Daten, die nicht aus der PostgreSQL oder dem Object Storage wiederherstellbar wären.
- Ergänzend werden regelmäßige Snapshots in den Object Storage geschrieben, um nach einem Neustart nicht mit vollständig leerem Cache zu starten.
- Ein Verlust bedeutet konkret: aktive Anmeldungen müssen sich neu anmelden, der Cache füllt sich neu, offene Hintergrundaufträge werden aus dem Datenbankstatus erneut eingestellt. Kein Verlust an Kurs-, Lern- oder Personendaten.
- **Restore:** Dienst neu starten, Worker verbinden sich automatisch neu; anschließend Nachlauf der offenen Aufträge.

SICHERHEITSASPEKTE

- **Nicht aus dem Internet erreichbar:** Dragonfly ist ausschließlich im internen Netz gebunden. Das ist der wichtigste Punkt, weil ungeschützt exponierte Redis-kompatible Dienste ein bekanntes Einfallstor sind.
- Zugriff nur mit Passwort, Verbindungen TLS-verschlüsselt.
- Ablaufzeiten auf allen Schlüsseln und eine feste Speichergrenze mit Verdrängungsstrategie begrenzen sowohl Datenhaltung als auch Ressourcenverbrauch.
- Im Speicher liegen Sitzungstoken und Job-Referenzen — keine Lernenden-Antworten, keine ILA-Konversationen, keine Stammdaten im Klartext.
- Snapshots werden verschlüsselt abgelegt.

8 GitHub

Quellcode, Reviews und automatisierte Auslieferung

WAS DIE KOMPONENTE MACHT

GitHub ist die Plattform, auf der der gesamte Quellcode der ILP versioniert verwaltet wird. Jede Änderung ist nachvollziehbar einer Person und einem Zeitpunkt zugeordnet und wird vor der Übernahme von einer zweiten Person geprüft. Über GitHub Actions laufen zusätzlich automatisierte Tests, Sicherheitsprüfungen und die Auslieferung in die Betriebsumgebung.

WIE IN DIE ILP INTEGRIERT

- Getrennte Repositories für Content Factory, Content Delivery, AI-Backend und Infrastruktur-Konfiguration.
- Jede Änderung durchläuft einen Pull Request mit Tests, Codeanalyse und Review — nichts gelangt ungeprüft in den Betrieb.
- Aus jedem freigegebenen Stand entsteht ein versioniertes Container-Image in der GitHub-Registry; genau dieses Image wird ausgerollt.
- Deployments laufen über GitHub Actions mit expliziter Freigabe für die Produktivumgebung.
- **In GitHub liegen keine personenbezogenen Daten** — kein Kursinhalt aus dem Betrieb, keine Lernenden-Daten, keine Zugangsdaten.

BACKUP & RESTORE

- Git ist von Natur aus verteilt: Jede Entwicklungsumgebung und jeder Build-Server hält eine vollständige Kopie der Historie.
- Zusätzlich läuft eine tägliche gespiegelte Sicherung aller Repositories inklusive Historie, Tags und Projektmetadaten auf eigene Infrastruktur.
- Container-Images der produktiv eingesetzten Versionen werden zusätzlich in der eigenen Umgebung vorgehalten, damit ein Rollout auch bei GitHub-Ausfall möglich bleibt.
- **Restore:** Bei Verlust eines Repositories wird aus der Spiegelung wiederhergestellt. Ein GitHub-Ausfall betrifft nur Entwicklung und Auslieferung, nicht den laufenden Lernbetrieb.

SICHERHEITSASPEKTE

- Zugang über die Unternehmens-Anmeldung mit verpflichtender Zwei-Faktor-Authentifizierung; Rechte werden über Teams nach dem Prinzip der geringsten Berechtigung vergeben, Austritte führen zum sofortigen Entzug.
- Geschützte Hauptbranches: kein direktes Schreiben, verpflichtendes Review, Tests müssen bestanden sein.
- **Secret Scanning** erkennt versehentlich eingecheckte Zugangsdaten sofort; produktive Zugangsdaten liegen grundsätzlich nicht in GitHub, sondern im Secret-Store der Betriebsumgebung.
- **Dependabot** meldet verwundbare Abhängigkeiten, **CodeQL** prüft den Code automatisiert auf Sicherheitsmängel.
- Fremde Actions werden nur in fest fixierten Versionen eingebunden, um Manipulation der Build-Kette auszuschließen.
- Deployments authentifizieren sich ohne dauerhaft hinterlegte Schlüssel; die Produktivfreigabe erfordert eine benannte Person.

Backup & Restore auf einen Blick

Was gesichert wird — und was bewusst nicht

Komponente	Was wird gesichert	Verfahren	Wiederherstellung
Rails-Backends	Nichts an der Anwendung; Daten in der PostgreSQL	Container-Image (edutech) + Datenbank- Backup (Group-IT)	Image ausrollen (Minuten); Rücksicherung der Datenbank über die Group-IT
FastAPI AI-Backend	Nichts; Code und Prompts in GitHub	Container-Image	Image ausrollen, offene Läufe neu anstoßen — Minuten
React-Frontends	Nichts; nur Quellcode	Build-Artefakt + GitHub	Vorherige Version ausrollen — Minuten
Celery	Warteschlange bewusst nicht	Auftragsstatus in der PostgreSQL	Worker starten, offene Aufträge neu einstellen
Sidekiq	Warteschlange bewusst nicht	Fachstatus in der PostgreSQL	Worker starten, Fehlerliste nacharbeiten
SolidQueue	Jobs liegen in der PostgreSQL	Automatisch im Datenbank-Backup der Group-IT	Worker starten; nach Rücksicherung Queue-Tabellen bereinigen
Dragonfly DB	Nur Snapshots des Caches	Snapshot in Object Storage	Dienst starten; kein Datenverlust mit Fachbezug
GitHub	Repositories mit voller Historie	Tägliche Spiegelung auf eigene Infrastruktur	Aus Spiegelung wiederherstellen; Lernbetrieb nicht betroffen

Kernaussage: Der gesamte schützenswerte Datenbestand der ILP liegt in der PostgreSQL und im Object Storage. Die PostgreSQL wird von der Group-IT betrieben und nach Konzernstandard gesichert; der Object Storage wird verschlüsselt und versioniert im ILP-Betrieb gesichert. Alle übrigen hier dokumentierten Komponenten sind austauschbare Laufzeit-Bausteine ohne eigenen Datenbestand, die im Ernstfall in Minuten neu ausgerollt werden.

SICHERHEIT — DIE WIEDERKEHRENDEN PRINZIPIEN

Prinzip	Umsetzung
Nur ausgereifte Standardtechnik	Alle Komponenten sind etablierte Open-Source-Projekte mit aktiver Wartung und großer Verbreitung. Keine Eigenentwicklung an sicherheitskritischen Grundfunktionen.
Nichts unnötig erreichbar	Aus dem Internet erreichbar sind nur die beiden Frontends und deren APIs. AI-Backend, alle Worker und Dragonfly sind ausschließlich intern erreichbar.
Daten bleiben, wo sie hingehören	Personenbezogene Daten liegen ausschließlich in der von der Group-IT betriebenen PostgreSQL. In Warteschlangen, Caches, Logs und GitHub stehen Referenzen statt Klartext.
Datenhaltung im Konzernbetrieb	Die Datenbank unterliegt dem Betrieb, der Härtung und den Sicherungsverfahren der Group-IT. Die ILP-Anwendungen greifen nur über eng berechnigte technische Konten aus dem internen Netz darauf zu.
Verschlüsselung durchgängig	TLS 1.3 im Transport, AES-256 auf den Datenträgern, Feldverschlüsselung für besonders sensible Inhalte wie ILA-Konversationen.
Zugangsdaten getrennt vom Code	Schlüssel und Passwörter liegen im Secret-Store der Betriebsumgebung, niemals im Repository. Automatische Erkennung versehentlicher Ausnahmen.
Jede Änderung nachvollziehbar	Vier-Augen-Prinzip über Pull Requests, automatisierte Sicherheitsprüfungen, Auslieferung nur aus geprüften Ständen.
Wiederholbarkeit statt Wiederherstellung	Hintergrundaufträge sind idempotent und aus dem fachlichen Zustand rekonstruierbar — der Verlust einer Warteschlange ist kein Datenverlust.

NOCH FESTZULEGEN

- Verbindliche RTO-/RPO-Zielwerte je Anwendung durch den Fachbereich — für die Datenbank abzugleichen mit den zugesagten Werten der Group-IT.
- Schriftliche Bestätigung der Group-IT zu Sicherungsfrequenz, Aufbewahrungsdauer und Wiederherstellungszeiten der PostgreSQL.
- Definierter Meldeweg für die Beauftragung eines Restores bei der Group-IT (Ansprechpartner, Erreichbarkeit außerhalb der Geschäftszeiten).
- Turnus und Protokollierung der gemeinsamen Restore-Tests (Vorschlag: halbjährlich in einer separaten Umgebung).
- Technologieentscheidung für die Lerndatenmessung — sobald getroffen, wird dieses Dokument entsprechend ergänzt.